WINDOW Engine

WINDOW Engine

Inhalt

1. 1. WINDOW Engine – Kompakte System-Referenz für KI-Modelle
2. 2. Operationen
3. 3. EXEC – Fenstersuche und Fensteroperationen
4. 4. ASSERT – Validierung
5. 5. BUFFER – Datenextraktion
6. 6. Verwendung von BUFFER-Werten
7. 7. Vollständiges Beispiel
8. 8. Troubleshooting & Best Practices
9. 9. Kurzreferenz

Die WINDOW Engine ermöglicht die Suche und Steuerung sichtbarer Desktop-Hauptfenster unter Windows. Sie unterstützt das Schließen, Maximieren, Minimieren und Wiederherstellen von Fenstern sowie die Validierung und Datenextraktion über ASSERT und BUFFER. Kontrollfluss wird durch das allgemeine condition-Feature und die vorhandene LOOP Engine gesteuert.

1. WINDOW Engine – Kompakte System-Referenz für KI-Modelle

```
# AGATE WINDOW-Engine Syntax-Referenz
# EXEC - Suche
- type: WINDOW
  op: EXEC
  action: FIND
  caption: 'e-card*'
  response: ecard_windows
  condition: "..."          # Optional

# EXEC - Fensteroperation
- type: WINDOW
  op: EXEC
  action: CLOSE             # CLOSE | MAXIMIZE | MINIMIZE | RESTORE
  caption: 'e-card*'
  condition: "..."          # Optional

# ASSERT - gespeichertes Ergebnis
- type: WINDOW
  op: ASSERT
  response: ecard_windows
  action: EXISTS            # EXISTS | COUNT | CAPTION | HANDLE |
PROCESS_ID
  expected: true
  index: 0                  # Optional bei CAPTION/HANDLE/PROCESS_ID
  condition: "..."          # Optional

# ASSERT - lebender Desktop mit Warten
- type: WINDOW
  op: ASSERT
  action: EXISTS
```

```
  caption: 'e-card*'
  expected: false
  wait: true
  timeout: 10000            # Millisekunden

# BUFFER
- type: WINDOW
  op: BUFFER
  response: ecard_windows
  action: EXISTS            # EXISTS | COUNT | CAPTION | HANDLE | PROCESS_ID
  index: 0                  # Optional, Default 0
  name: ecard_exists
  condition: "..."          # Optional
```

2. Operationen

Operation	Beschreibung
EXEC	Sucht Desktop-Fenster oder führt eine Fensteroperation aus.
ASSERT	Validiert ein gespeichertes Suchergebnis oder prüft den lebenden Desktop.
BUFFER	Extrahiert einen Wert aus einem zuvor gespeicherten Suchergebnis.

Die Engine folgt dem **Decoupled Execution Model**: FIND führt eine Suche aus und speichert den Ergebnis-Snapshot unter response. ASSERT und BUFFER können denselben Snapshot mehrfach auswerten, ohne die Suche zu wiederholen.

3. EXEC – Fenstersuche und Fensteroperationen

3.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
action	FIND, CLOSE, MAXIMIZE, MINIMIZE oder RESTORE	Ja	–
caption	Titelmuster der gesuchten Fenster	Ja	–
response	Speichername des	Bei FIND	–

	Suchergebnisse s		
condition	Allgemeine AGATE-Ausführungsbedingung	Nein	–

```yaml
- id: find_ecard
  type: WINDOW
  op: EXEC
  action: FIND
  caption: 'bank-card*'
  response: ecard_windows
```

3.2 Caption und Wildcards

Muster	Bedeutung
bank-card*	Titel beginnt mit e-card.
Google	Titel enthält Google.
Test?	Test gefolgt von genau einem Zeichen.
Mein Fenster	Exakter Titel, ohne Beachtung der Groß-/Kleinschreibung.

Das Matching ignoriert Groß-/Kleinschreibung. * und ? sind die einzigen Wildcards; andere Zeichen werden wörtlich behandelt. Es werden sichtbare Hauptfenster erfasst, keine HTML-Elemente und keine einzelnen Browser-Tabs.

3.3 Suchergebnis und response

response ist ein frei wählbarer Speichername, kein Dateipfad. Intern wird ein unveränderlicher WindowResult mit einer Liste WindowInfo gespeichert. Die entsprechende JSON-Darstellung lautet:

```json
{
  "windows": [
    {
      "handle": "123456",
      "caption": "bank-card - Anmeldung",
      "processId": 9876
    }
  ]
}
```

Feld	Bedeutung

handle	Technische Kennung des Desktop-Fensters, als Zeichenkette.
caption	Tatsächlicher Fenstertitel.
processId	PID des Prozesses, dem das Fenster gehört.

Wenn kein Fenster passt, enthält windows eine leere Liste. FIND ist dann erfolgreich; EXISTS liefert false. Ein technischer Fehler beendet den Schritt mit Fehler und wird nicht als leeres Ergebnis behandelt.

```
{"windows": []}
```

Der Snapshot aktualisiert sich nicht selbst. Ein erneutes FIND unter demselben response ersetzt den gespeicherten Snapshot durch das aktuelle Ergebnis.

3.4 CLOSE, MAXIMIZE, MINIMIZE und RESTORE

Action	Verhalten
CLOSE	Sendet eine Schließanforderung an alle aktuell passenden Fenster.
MAXIMIZE	Maximiert alle aktuell passenden Fenster.
MINIMIZE	Minimiert alle aktuell passenden Fenster.
RESTORE	Stellt alle aktuell passenden Fenster wieder her.

```
- id: close_services_windows
  type: WINDOW
  op: EXEC
  action: CLOSE
  caption: 'services-*'

- id: maximize_ecard
  type: WINDOW
  op: EXEC
  action: MAXIMIZE
  caption: 'bank-card*'
```

Diese Aktionen suchen beim Ausführen erneut nach caption. Ohne Treffer endet der Schritt erfolgreich ohne Änderung. CLOSE fordert das Schließen an; eine Anwendung kann anschließend noch eine Rückfrage anzeigen. Für eine garantierte Zustandsprüfung folgt ein neues FIND oder eine wartende ASSERT-Prüfung.

3.5 Timeout und Voraussetzungen

WINDOW benötigt eine interaktive Windows-Sitzung und Windows PowerShell. Der Desktop-Provider hat pro PowerShell-Aufruf ein festes technisches Limit von 30 Sekunden. Das YAML-Feld timeout steuert in dieser Version die wartende ASSERT-Prüfung in Millisekunden; es konfiguriert kein EXEC-Timeout.

Im Unterschied zur CMD-Referenz ist WINDOW ASSERT timeout in **Millisekunden** angegeben. Ein laufender Desktop-Aufruf kann nicht durch den kürzeren ASSERT-Wert vorzeitig unterbrochen werden.

4. ASSERT – Validierung

ASSERT vergleicht den ausgewerteten Istwert mit expected. Eine fehlgeschlagene Prüfung führt zu einem fehlgeschlagenen Testschritt.

Action	Beschreibung	Zusätzliche Parameter
EXISTS	Prüft, ob mindestens ein Treffer vorhanden ist.	expected: true oder false
COUNT	Prüft die Anzahl der Treffer.	expected
CAPTION	Prüft den Titel eines Treffers.	expected, index optional
HANDLE	Prüft die Fensterkennung eines Treffers.	expected, index optional
PROCESS_ID	Prüft die PID eines Treffers.	expected, index optional

EXISTS

```
- type: WINDOW
  op: ASSERT
  response: ecard_windows
  action: EXISTS
  expected: true
```

COUNT

```
- type: WINDOW
  op: ASSERT
  response: ecard_windows
  action: COUNT
  expected: '1'
```

CAPTION

```yaml
- type: WINDOW
  op: ASSERT
  response: ecard_windows
  action: CAPTION
  index: 0
  expected: 'bank-card - Anmeldung'
```

COUNT, CAPTION, HANDLE und PROCESS_ID werden als Zeichenketten exakt verglichen; es gibt hier keine Teiltext-, Glob- oder Regex-Assertion. EXISTS akzeptiert ausschließlich einen booleschen Erwartungswert true/false.

Auf einen aktuellen Zustand warten

```yaml
- type: WINDOW
  op: ASSERT
  action: EXISTS
  caption: 'services-*'
  expected: false
  wait: true
  timeout: 10000
```

Ohne response wird der Desktop über caption direkt geprüft. wait: true wiederholt die Live-Abfrage bis zum erwarteten Zustand. Standard für timeout ist 30000 Millisekunden. wait: true ist nur mit EXISTS und caption erlaubt; ein gespeicherter Snapshot kann durch Warten nicht aktualisiert werden.

5. BUFFER – Datenextraktion

BUFFER extrahiert Informationen aus einem zuvor gespeicherten FIND-Ergebnis und speichert den Wert unter name.

```yaml
- type: WINDOW
  op: BUFFER
  response: bankcard_windows
  action: EXISTS
  name: bankcard_exists
```

Der erzeugte Wert wird in späteren Schritten über {B[bankcard_exists]} verwendet. Für name sollte ein anderer Bezeichner als für response gewählt werden, damit das ursprüngliche Suchergebnis nicht überschrieben wird.

5.1 BUFFER Actions

Action	Beschreibung	Parameter
EXISTS	Speichert true oder false als Zeichenkette.	–
COUNT	Speichert die Trefferzahl als Zeichenkette.	–
CAPTION	Extrahiert den Fenstertitel.	index optional, Default 0

HANDLE	Extrahiert die Fensterkennung.	index optional, Default 0
PROCESS_ID	Extrahiert die PID.	index optional, Default 0

EXISTS

```
- type: WINDOW
  op: BUFFER
  response: ecard_windows
  action: EXISTS
  name: bankcard_exists
```

COUNT

```
- type: WINDOW
  op: BUFFER
  response: ecard_windows
  action: COUNT
  name: bankcard_count
```

CAPTION

```
- type: WINDOW
  op: BUFFER
  response: bankcard_windows
  action: CAPTION
  index: 0
  name: ecard_caption
```

HANDLE

```
- type: WINDOW
  op: BUFFER
  response: bankcard_windows
  action: HANDLE
  index: 0
  name: bankcard_handle
```

PROCESS_ID

```
- type: WINDOW
  op: BUFFER
  response: bankcard_windows
  action: PROCESS_ID
  index: 0
  name: ecard_pid
```

5.2 Bedeutung von index

index beginnt bei 0: 0 ist der erste Treffer, 1 der zweite und 2 der dritte. Die Reihenfolge ist nicht fachlich garantiert und sagt nichts über das aktive oder zuerst geöffnete Fenster aus. Für eine eindeutige Auswahl sollte caption so genau wie möglich formuliert werden.

EXISTS und COUNT funktionieren bei einer leeren Liste. CAPTION/HANDLE/PROCESS_ID benötigen einen gültigen Trefferindex. Ein fehlender response, ein falscher Ergebnistyp oder ein ungültiger Index führt zu einem Fehler.

5.3 Verwendung von HANDLE und PROCESS_ID

HANDLE und PROCESS_ID können gespeichert und validiert werden. EXEC-Aktionen per Handle/PID sowie KILL_PROCESS sind in dieser Version noch nicht implementiert. Ein PID-Abbruch würde den Prozess betreffen und kann mehrere Fenster beeinflussen; CLOSE beendet deshalb niemals automatisch einen Prozess.

6. Verwendung von BUFFER-Werten

```yaml
- id: find_ecard
  type: WINDOW
  op: EXEC
  action: FIND
  caption: 'bank-card*'
  response: bankcard_windows

- id: extract_exists
  type: WINDOW
  op: BUFFER
  response: bankcard_windows
  action: EXISTS
  name: bankcard_exists

- id: close_only_when_present
  type: WINDOW
  condition: "'{B[bankcard_exists]}' == 'true'"
  op: EXEC
  action: CLOSE
  caption: 'bank-card*'
```

Wenn bankcard_exists false ist, wird CLOSE als SKIPPED behandelt. Ein ASSERT EXISTS mit expected true wäre für eine optionale Aktion ungeeignet: Es würde bei fehlendem Fenster den Test fehlschlagen lassen.

6.1 Wiederholung mit LOOP

```yaml
- id: close_services_repeatedly
  type: LOOP
  mode: DO_WHILE
  maxIterations: 10
  timeoutMs: 60000
  condition: "'{B[services_exists]}' == 'true'"
  steps:

    - id: close_services
      type: WINDOW
      op: EXEC
      action: CLOSE
      caption: 'services-*'

    - id: wait_for_close
      type: WAIT
      value: '2000'

    - id: find_remaining_services
      type: WINDOW
      op: EXEC
      action: FIND
      caption: 'services-*'
      response: services_windows

    - id: update_loop_condition
      type: WINDOW
      op: BUFFER
      response: services_windows
```

```
      action: EXISTS
      name: services_exists
```

DO_WHILE führt zuerst den Rumpf aus. FIND/BUFFER am Ende bestimmen, ob der nächste Durchlauf nötig ist. WHILE benötigt diese Abfrage zusätzlich vor dem ersten Durchlauf. Es gibt keine WINDOW-Operationen IF/WHILE/DO_WHILE: condition und LOOP bleiben globale AGATE-Funktionen.

7. Vollständiges Beispiel

Das Beispiel prüft eine optionale Desktop-Fenstersuche. Ohne Treffer bleibt der Test erfolgreich; vorhandene Treffer werden ausgewertet. Es schließt keine externen Fenster.

```yaml
testCases:
  - id: TC_WINDOW_Engine_Demo
    description: 'WINDOW Engine - Suche, BUFFER und ASSERT'
    stage: '*'
    priority: HIGH
    variables: {}
    steps:

      - id: find_google_windows
        type: WINDOW
        op: EXEC
        action: FIND
        caption: '*Google*'
        response: google_windows

      - id: extract_exists
        type: WINDOW
        op: BUFFER
        response: google_windows
        action: EXISTS
        name: google_window_exists

      - id: extract_count
        type: WINDOW
        op: BUFFER
        response: google_windows
        action: COUNT
        name: google_window_count

      - id: extract_first_caption
        type: WINDOW
        condition: "'{B[google_window_exists]}' == 'true'"
        op: BUFFER
        response: google_windows
        action: CAPTION
        index: 0
        name: google_first_caption

      - id: extract_first_pid
        type: WINDOW
        condition: "'{B[google_window_exists]}' == 'true'"
        op: BUFFER
        response: google_windows
        action: PROCESS_ID
        index: 0
        name: google_first_pid

      - id: verify_saved_result
        type: WINDOW
        condition: "'{B[google_window_exists]}' == 'true'"
        op: ASSERT
        response: google_windows
        action: EXISTS
```

```
54          expected: true
```

8. Troubleshooting & Best Practices

Kein Fenster gefunden

Prüfe den tatsächlichen Desktop-Fenstertitel, die Wildcards und die aktive Windows-Sitzung. Die Suche erfasst sichtbare Hauptfenster. Ein Browser-Tab mit Google im Inhalt muss nicht zwingend einen sichtbaren Hauptfenstertitel mit Google erzeugen.

ASSERT meldet weiterhin EXISTS=true nach CLOSE

Ein alter response bleibt ein Snapshot vor dem Schließen. Führe FIND erneut aus oder verwende ASSERT mit caption und wait: true. Eine Schließanforderung kann außerdem durch einen Bestätigungsdialog verzögert werden.

Index liegt außerhalb der Trefferliste

Prüfe zuerst EXISTS oder COUNT. index 0 setzt mindestens einen Treffer voraus. Mehrere Treffer sollten möglichst durch ein genaueres caption-Muster vermieden werden.

Condition wird übersprungen

Boolesche BUFFER-Werte werden als Zeichenketten true/false gespeichert. Nutze einen String-Vergleich:

```
1 condition: "'{B[my_window_exists]}' == 'true'"
```

Browser oder einzelner Tab?

Ziel	Passende Aktion
Benannten Tab öffnen	GUI / EXEC / OPEN_TAB mit tab und url
Benannten Tab suchen	GUI / EXEC / FIND_TAB
Einzelnen benannten Tab schließen	GUI / EXEC / CLOSE_TAB
AGATE-Browser samt allen Tabs schließen	GUI / EXEC / CLOSE_BROWSER
Desktop-Fenster schließen	WINDOW / EXEC / CLOSE mit caption

Die GUI-Tab-Aktionen betreffen ausschließlich die von AGATE verwaltete Sitzung. FIND_TAB durchsucht in dieser Version benannte Tabs aus OPEN_TAB. Es hängt sich nicht an einen privat gestarteten Browser an. WINDOW arbeitet dagegen mit Desktop-Hauptfenstern der aktuellen Windows-Sitzung.

Noch nicht unterstützte Funktionen

Native UIA-/AnyUI-Unterelemente, locatorRef-Bindings, Edge-Wiederherstellen-Popup, native Send Keys, KILL_PROCESS und Fensteraktionen per Handle/PID sind noch nicht unterstützt. Nicht unterstützte Aktionen werden ausdrücklich als Fehler gemeldet.

9. Kurzreferenz

Bereich	Syntax / Verhalten
EXEC FIND	caption + response → WindowResult-Snapshot
EXEC CLOSE/MAXIMIZE/MINIMIZE/RESTORE	caption → Aktion auf aktuelle Treffer
BUFFER EXISTS/COUNT	response + name
BUFFER CAPTION/HANDLE/PROCESS_ID	response + name + optional index (0-basiert)
ASSERT	action + expected + response oder caption
ASSERT wait	EXISTS + caption + expected + wait:true + timeout in ms
condition	Allgemeine AGATE-Ausführungsbedingung
LOOP	WHILE/DO_WHILE + condition + maxIterations + timeoutMs
Fenster nicht vorhanden	FIND erfolgreich; EXISTS=false; COUNT=0
Mehrere Treffer	EXEC spricht alle Treffer an; BUFFER mit index wählt einen Snapshot-Treffer